Assignment 8

Name: Riddhi Agarwal
Roll Number: 2024CSB046

Problem 1

Code

```c
#include <stdio.h>
#include <time.h>

unsigned long long fibonacci_recursive(int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci_recursive(n - 1) + fibonacci_recursive(n - 2);
}

unsigned long long fibonacci_dp(int n) {
    if (n <= 1) {
        return n;
    }

    unsigned long long dp[n + 1];
    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }

    return dp[n];
}

int main() {
    int n;
    printf("Enter the value of n: ");
    scanf("%d", &n);

    clock_t start, end;
    double cpu_time_used;

    printf("\n--- Dynamic Programming Method ---\n");
    start = clock();
    unsigned long long result_dp = fibonacci_dp(n);
    end = clock();
    cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
```

```c
    printf("Fibonacci(%d) = %llu\n", n, result_dp);
    printf("Execution Time: %f seconds\n", cpu_time_used);

    printf("\n--- Recursive Method ---\n");
    start = clock();
    unsigned long long result_rec = fibonacci_recursive(n);
    end = clock();
    cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
    printf("Fibonacci(%d) = %llu\n", n, result_rec);
    printf("Execution Time: %f seconds\n", cpu_time_used);

    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment8> gcc .\Ques1.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment8> ./a.exe
Enter the value of n: 44

--- Dynamic Programming Method ---
Fibonacci(44) = 701408733
Execution Time: 0.000000 seconds

--- Recursive Method ---
Fibonacci(44) = 701408733
Execution Time: 7.902000 seconds
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment8>
```

Figure 1: Output for Problem 1.

Comparative Analysis

Time Complexity

- **Recursive Method:** The time complexity is exponential, $O(2^n)$, in the best, average, and worst cases. This is because the algorithm repeatedly calculates the same Fibonacci numbers over and over again, leading to a massive tree of redundant recursive calls.
- **Dynamic Programming Method:** The time complexity is strictly linear, $O(n)$, across the best, average, and worst cases. The algorithm simply iterates from 2 up to n exactly once.

Space Complexity

- **Recursive Method:** The space complexity is $O(n)$ in all cases. This memory is consumed by the call stack, which reaches a maximum depth of n during the deepest recursive branch.
- **Dynamic Programming Method:** Using the standard tabulation method (as shown in the code), the space complexity is $O(n)$ to maintain the array that stores all previously calculated values.

Execution Performance

When the number of retries (or n) becomes large, the recursive approach is extremely slow. If you test the program above with an n of 45 or 50, you will notice the recursive method hangs for a significant amount of time, while the DP method finishes almost instantly. The DP approach is highly efficient for large Fibonacci calculations because it reuses stored values.

Problem 2

Code

```c
#include <stdio.h>
#include <limits.h>
#include <time.h>

int matrixChainRecursive(int p[], int i, int j) {
    if (i == j) {
        return 0;
    }

    int k;
    int min = INT_MAX;
    int count;

    for (k = i; k < j; k++) {
        count = matrixChainRecursive(p, i, k) +
                matrixChainRecursive(p, k + 1, j) +
                p[i - 1] * p[k] * p[j];

        if (count < min) {
            min = count;
        }
    }
    return min;
}

int matrixChainDP(int p[], int n) {
    int m[n][n];
    int i, j, k, L, q;

    for (i = 1; i < n; i++) {
        m[i][i] = 0;
    }

    for (L = 2; L < n; L++) {
        for (i = 1; i < n - L + 1; i++) {
            j = i + L - 1;
            m[i][j] = INT_MAX;
            for (k = i; k <= j - 1; k++) {
                q = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];
                if (q < m[i][j]) {
                    m[i][j] = q;
                }
            }
        }
    }

    return m[1][n - 1];
```

```c
}

int main() {
    int arr[] = {10, 30, 5, 60};
    int n = sizeof(arr) / sizeof(arr[0]);

    clock_t start, end;
    double cpu_time_used;

    printf("Matrix Dimensions Array: ");
    for(int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n\n");

    printf("--- Dynamic Programming Approach ---\n");
    start = clock();
    int dp_result = matrixChainDP(arr, n);
    end = clock();
    cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
    printf("Minimum number of scalar multiplications: %d\n", dp_result);
    printf("Execution Time: %f seconds\n\n", cpu_time_used);

    printf("--- Recursive Approach ---\n");
    start = clock();
    int rec_result = matrixChainRecursive(arr, 1, n - 1);
    end = clock();
    cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;
    printf("Minimum number of scalar multiplications: %d\n", rec_result);
    printf("Execution Time: %f seconds\n", cpu_time_used);

    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment8> gcc .\Ques2.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment8> ./a.exe
Matrix Dimensions Array: 10 30 5 60

--- Dynamic Programming Approach ---
Minimum number of scalar multiplications: 4500
Execution Time: 0.000000 seconds

--- Recursive Approach ---
Minimum number of scalar multiplications: 4500
Execution Time: 0.000000 seconds
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment8>
```

Figure 2: Output for Problem 2.

Comparative Analysis

Time Complexity

- **Recursive Approach:** The time complexity is exponential, specifically $O(2^n)$, across the best, average, and worst cases. This is because the algorithm repeatedly calculates the exact same subproblems (overlapping subproblems) while checking all possible ways of multiplying the sequence.
- **Dynamic Programming Approach:** The time complexity is $O(n^3)$ in the best, average, and worst cases. The algorithm iterates through chain lengths, start indices, and partition points using three nested loops, resolving the overlapping subproblem issue by reading from the table.

Space Complexity

- **Recursive Approach:** The space complexity is $O(n)$ in all cases. This memory is utilized purely by the call stack for the recursive function calls, which reaches a maximum depth of n.
- **Dynamic Programming Approach:** The space complexity is $O(n^2)$ in all cases. The standard DP approach requires an $n \times n$ 2D array (the table) to store intermediate calculation results.

Execution Performance

When the number of matrices in the sequence is small (like the 3 matrices in the assignment example), both methods will execute nearly instantaneously. However, as the number of matrices increases, the execution time for the recursive method will degrade rapidly due to its exponential nature. The dynamic programming approach will remain highly efficient for larger sequences because it avoids recalculating identical sub-chains.

Problem 3

Code

```c
#include <stdio.h>
#include <time.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int knapsackRecursive(int W, int wt[], int val[], int n) {
    if (n == 0 || W == 0) {
        return 0;
    }

    if (wt[n - 1] > W) {
        return knapsackRecursive(W, wt, val, n - 1);
    }
    else {
        return max(
            val[n - 1] + knapsackRecursive(W - wt[n - 1], wt, val, n - 1),
            knapsackRecursive(W, wt, val, n - 1)
        );
    }
}

int knapsackDP(int W, int wt[], int val[], int n) {
    int i, w;
    int K[n + 1][W + 1];

    for (i = 0; i <= n; i++) {
        for (w = 0; w <= W; w++) {
            if (i == 0 || w == 0) {
                K[i][w] = 0;
            } else if (wt[i - 1] <= w) {
                K[i][w] = max(val[i - 1] + K[i - 1][w - wt[i - 1]], K[i - 1][w]);
            } else {
                K[i][w] = K[i - 1][w];
            }
        }
    }
    return K[n][W];
}

int main() {
    int val[] = {60, 100, 120, 90, 50};
    int wt[] = {10, 20, 30, 15, 5};
    int W = 50;
```

```c
    int n = sizeof(val) / sizeof(val[0]);

    clock_t start, end;
    double cpu_time_used;

    printf("Number of packages: %d\n", n);
    printf("Truck Capacity: %d\n\n", W);

    printf("--- Dynamic Programming Approach ---\n");
    start = clock();
    int dp_result = knapsackDP(W, wt, val, n);
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Maximum Profit: %d\n", dp_result);
    printf("Execution Time: %f seconds\n\n", cpu_time_used);

    printf("--- Recursive Approach ---\n");
    start = clock();
    int rec_result = knapsackRecursive(W, wt, val, n);
    end = clock();
    cpu_time_used = ((double)(end - start)) / CLOCKS_PER_SEC;
    printf("Maximum Profit: %d\n", rec_result);
    printf("Execution Time: %f seconds\n", cpu_time_used);

    return 0;
}
```

Output

```
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment8> gcc .\Ques3.c
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment8> ./a.exe
Number of packages: 5
Truck Capacity: 50

--- Dynamic Programming Approach ---
Maximum Profit: 300
Execution Time: 0.000000 seconds

--- Recursive Approach ---
Maximum Profit: 300
Execution Time: 0.000000 seconds
PS C:\Users\USER\code\college-lab-work\Semester4\AlgorithmLab\Assignment8>
```

Figure 3: Output for Problem 3.

Comparative Analysis

Time Complexity

- **Recursive Approach:** The time complexity is $O(2^n)$ in the best, average, and worst cases. The algorithm evaluates every possible combination of including or excluding each package, leading to an exponential number of recursive calls.

- **Dynamic Programming Approach:** The time complexity is $O(n \times W)$ in the best, average, and worst cases (where n is the number of packages and W is the maximum weight capacity). The algorithm uses nested loops to populate a table of size $(n + 1) \times (W + 1)$ exactly once.

Space Complexity

- **Recursive Approach:** The space complexity is $O(n)$ across all cases. This memory is entirely consumed by the function call stack, which reaches a maximum depth equal to the total number of packages n.
- **Dynamic Programming Approach:** The space complexity is $O(n \times W)$ in all cases because it requires an $(n + 1) \times (W + 1)$ 2D array to store the intermediate calculation results to avoid recalculating them.

Efficiency for Large Number of Packages

When the number of packages becomes very large, checking all combinations via the recursive approach becomes incredibly slow and practically unusable due to exponential time growth. Conversely, the dynamic programming approach remains highly efficient and executes quickly, provided the maximum weight capacity W isn't astronomically large (as its performance is pseudo-polynomial, dependent on both n and W).